

13-06-2026

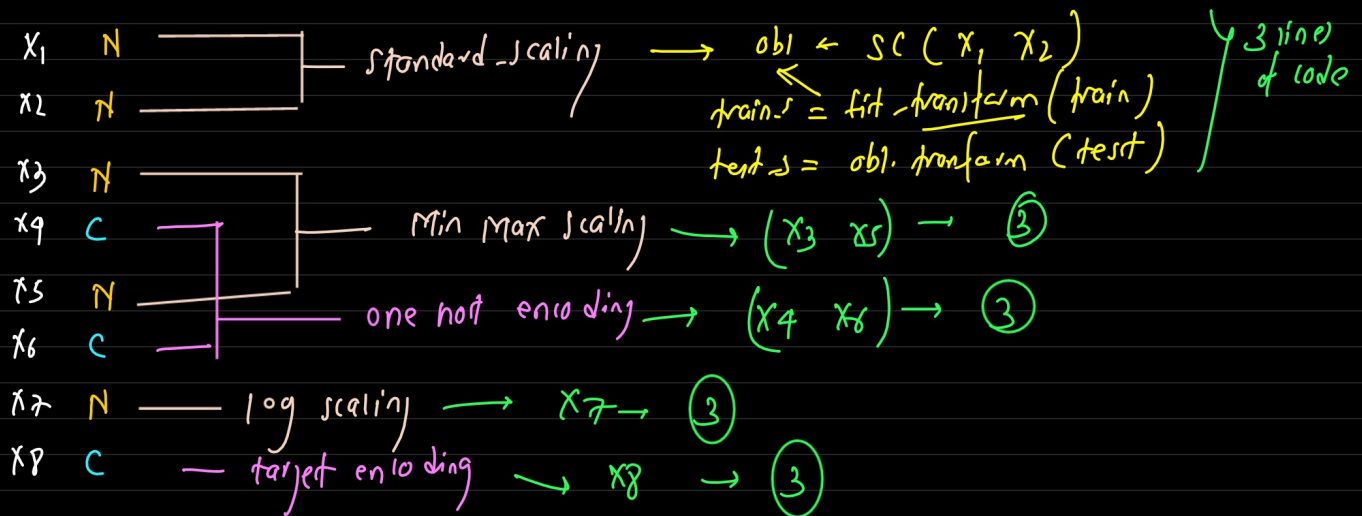
Agenda:

ML-6

- Linear Regression → Testing ✓
- Regularization
- Logistic Regression (Algo-2)

Column Transformer

train test

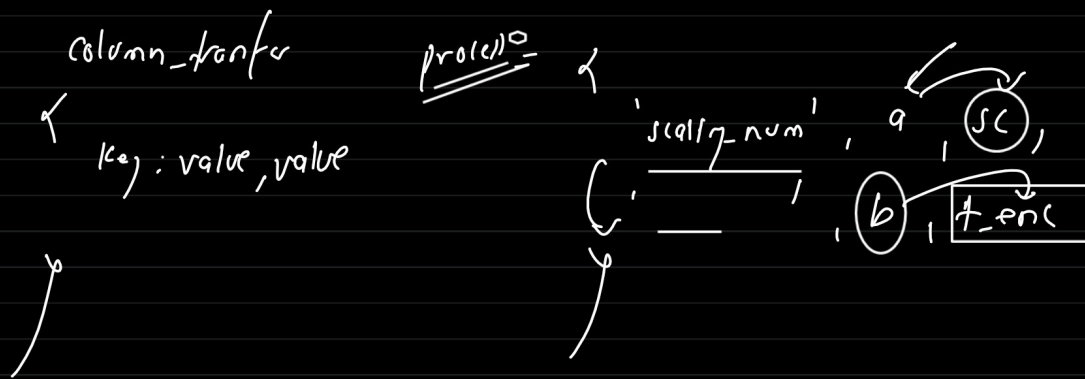


total → 15 lines of code

with column transformer:

$[x_1, x_2] \rightarrow a$
 $[x_3, x_5] \rightarrow b$
 $[x_4, x_6] \rightarrow c$
 $[x_7] \rightarrow d$
 $[x_8] \rightarrow e$

$sc = StandardScaler()$
 $m = min_max$
 $ohc = OneHotEncoder()$
 $lsc = log_scaling()$
 $t_enc = target_encoder()$



preprocessor.fit_transform(X_train)

A, B, C, D, E, F

```
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer, categorical_features),
    ],
    remainder='passthrough'
)
```

↓

C

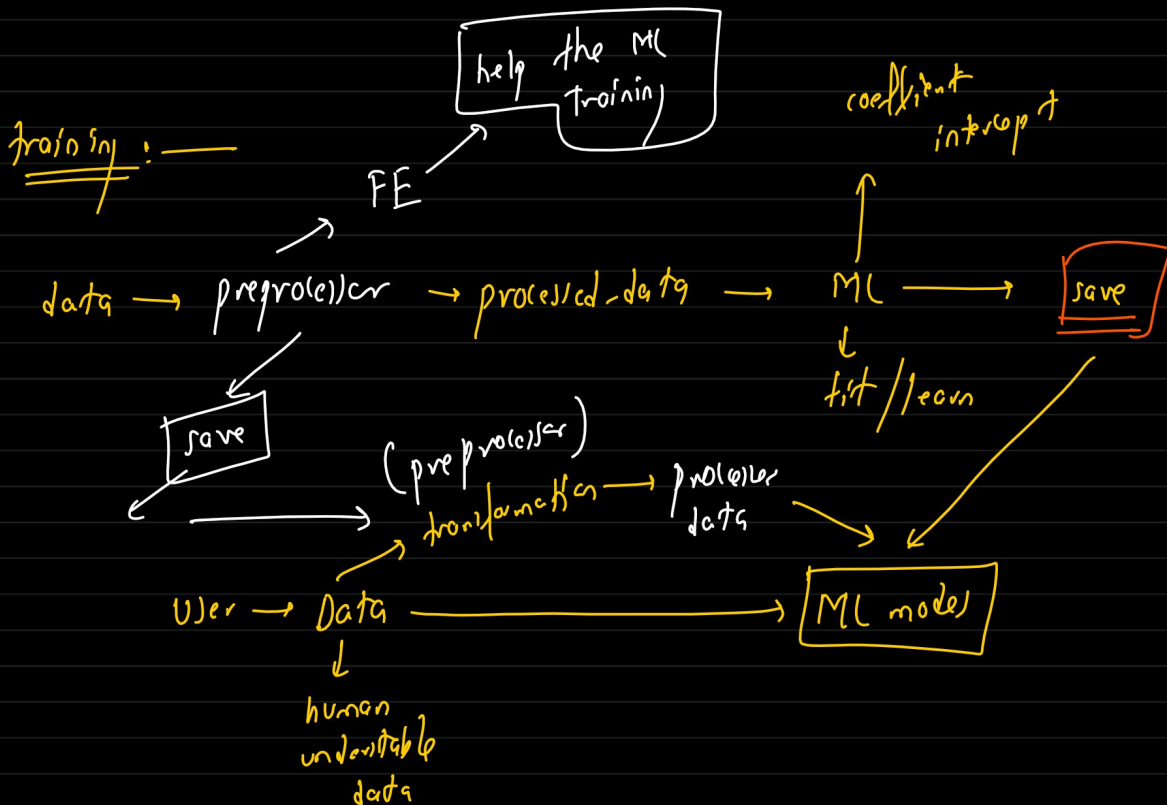
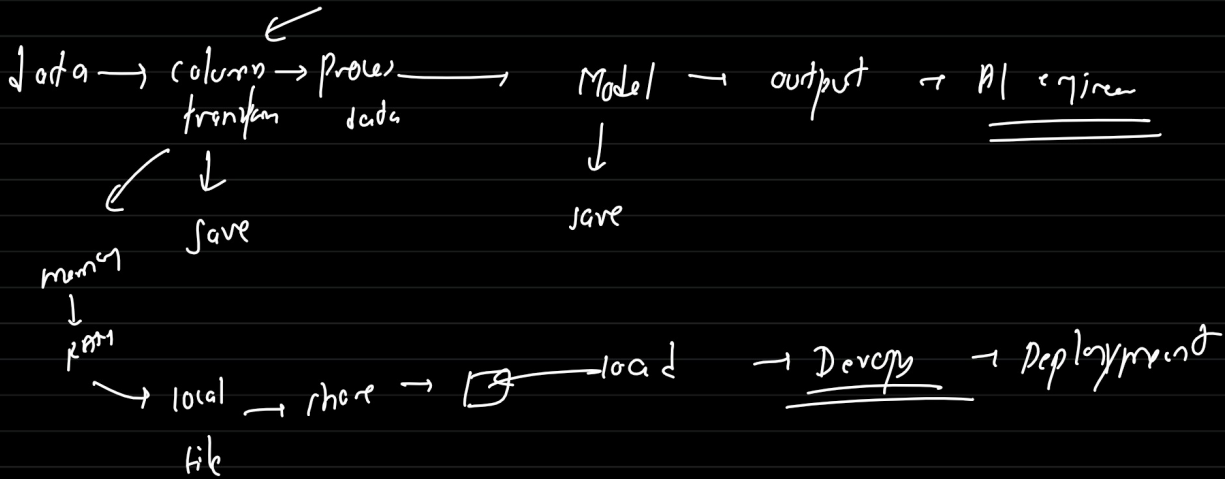
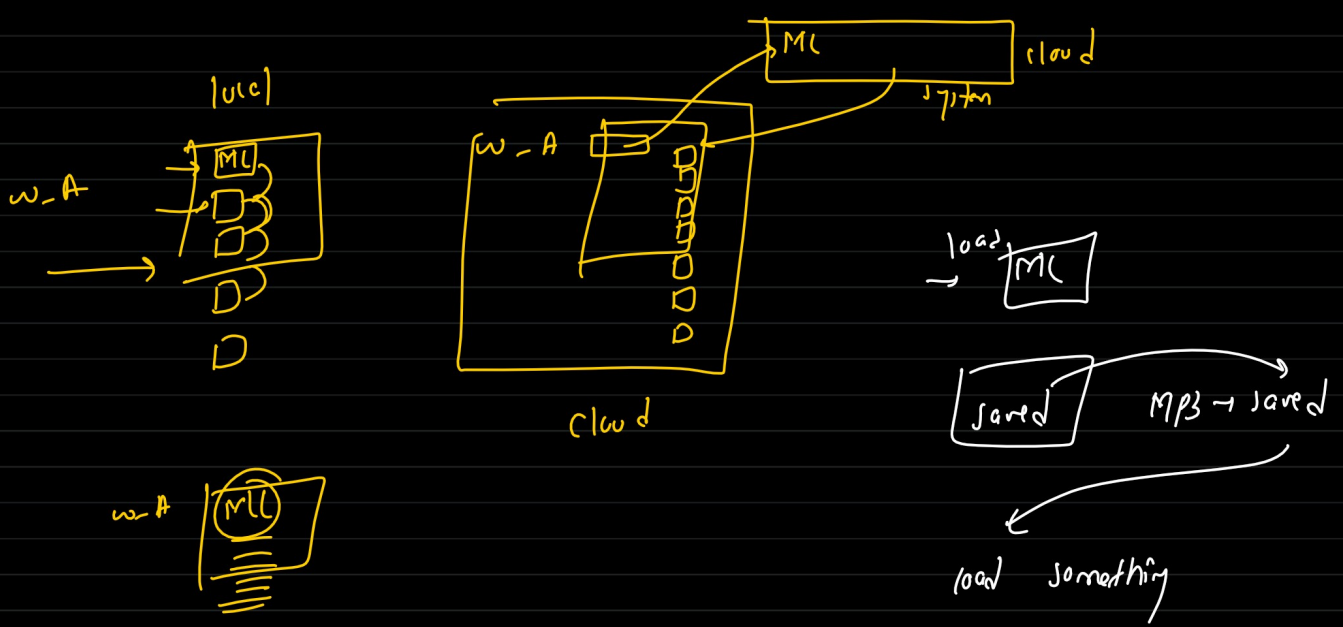
↓

keep it in the set

preprocessor(X_train)

A, B, C, D, E, F

weather_forecast → input → ML →



ML $\rightarrow$ output file

EDA $\rightarrow$ report $\rightarrow$.pdf, .html

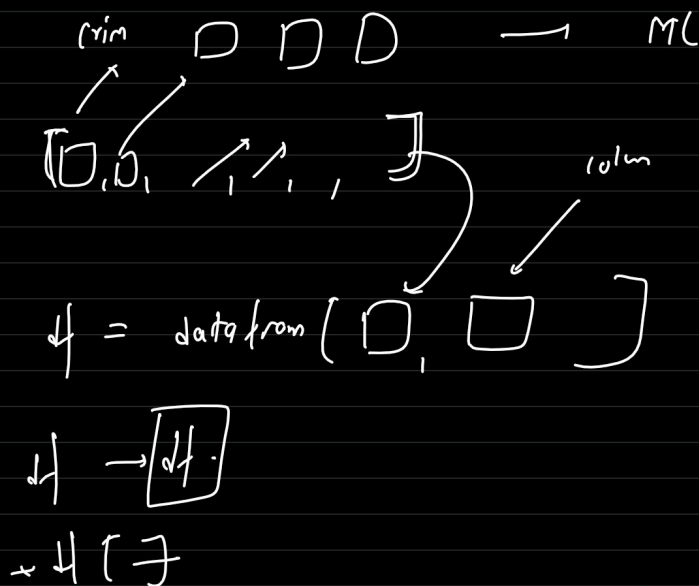
$\rightarrow$ plot visualization $\rightarrow$.png, .jpg

ML $\rightarrow$ model.joblib

$\rightarrow$ preprocessor.joblib

scikit-learn object $\rightarrow$ joblib $\rightarrow$

python modules/object $\rightarrow$ pickle



linear_regression_test.py $\leftarrow$ optimize code

keep the prediction function in ∞ while loop

Assignment-1

Assignment-2

dry saving the file using pickle &
load it in ^{output} prediction

$\downarrow$
in loop as user

want prediction? Y or N

YY nN $\rightarrow$ NO skip

Yes $\rightarrow$ prediction

Regularization

$x_1 \quad x_2 \quad x_3 \quad y$

1 200 1 2000

$$\rightarrow m_1 x_1 + m_2 x_2 + m_3 x_3 + C$$

$m_1 \quad m_2 \quad m_3 \quad C$

→ case-3 → hardest

→ case-1 → simplest

case-1 → $m_1 \rightarrow 0$

$m_2 \rightarrow 10$

$m_3 \rightarrow 0$

$C \rightarrow 0$

case-2 → $m_1 \rightarrow 1000$

$m_2 \rightarrow 0$

$m_3 \rightarrow 1000$

$C \rightarrow 0$

case-3 → $m_1 \rightarrow 4000$

$m_2 \rightarrow -10$

$m_3 \rightarrow 0$

$C \rightarrow 0$

case-4 → $m_1 \rightarrow 0$

$m_2 \rightarrow 0$

$m_3 \rightarrow 0$

$C \rightarrow 2000$

case-3, small random value of β
 $|r = 0.01$

↓

case-1 → fastest

case-5 → $m_1 \rightarrow 500$

$m_2 \rightarrow 5$

$m_3 \rightarrow 500$

$C \rightarrow 0$

case-6 → $m_1 \rightarrow 4000$

→ $m_2 \rightarrow -10$

→ $m_3 \rightarrow +999999 \times 1$

$m_4 \rightarrow -999999 \times 1$

large $\beta(m_0, m_1, m_2) \rightarrow$ stuck in loop

→ computation

→ 999 → -1000
 → -999 → 1000

Don't just minimize error, keep the coefficients controlled.

$x_1 \quad x_2 \quad x_3 \quad y$

1 200 1 2000 $\rightarrow$ $m_1 x_1 + m_2 x_2 + m_3 x_3 + c$

linear regression : —

$\beta \leftarrow$ initialise random $\leftarrow$ small β
 $\leftarrow$ large β

$x_1 m_1 + x_2 m_2 + x_3 m_3 + c \rightarrow \hat{y}$

iter : (1) (2) (3) $\rightarrow$ 4000 steps

$m_1 \rightarrow 1$ $m_1 \rightarrow 2$ $m_1 \rightarrow 3$
 $m_2 \rightarrow 0$ $m_2 \rightarrow 0$ $m_2 \rightarrow 0$
 $m_3 \rightarrow 0$ $m_3 \rightarrow 0$ $m_3 \rightarrow 0$
 $c \rightarrow$ $c \rightarrow 0$ $c \rightarrow 0$

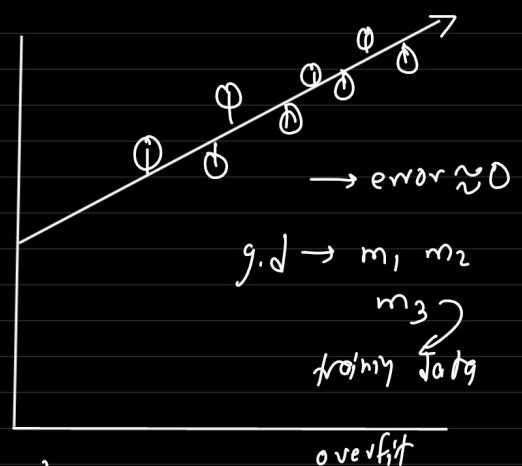
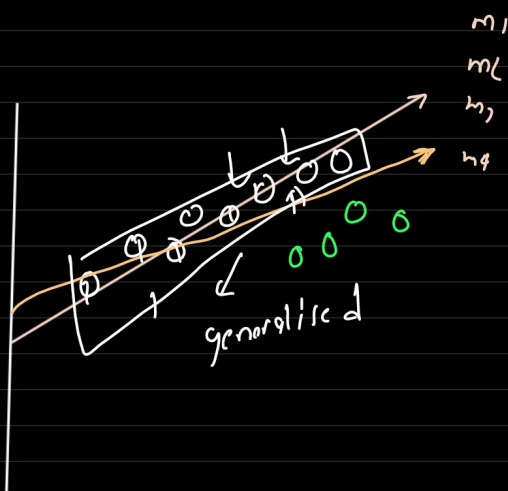
$\rightarrow$ CD
 $lr \rightarrow \uparrow$

(1) $\uparrow$

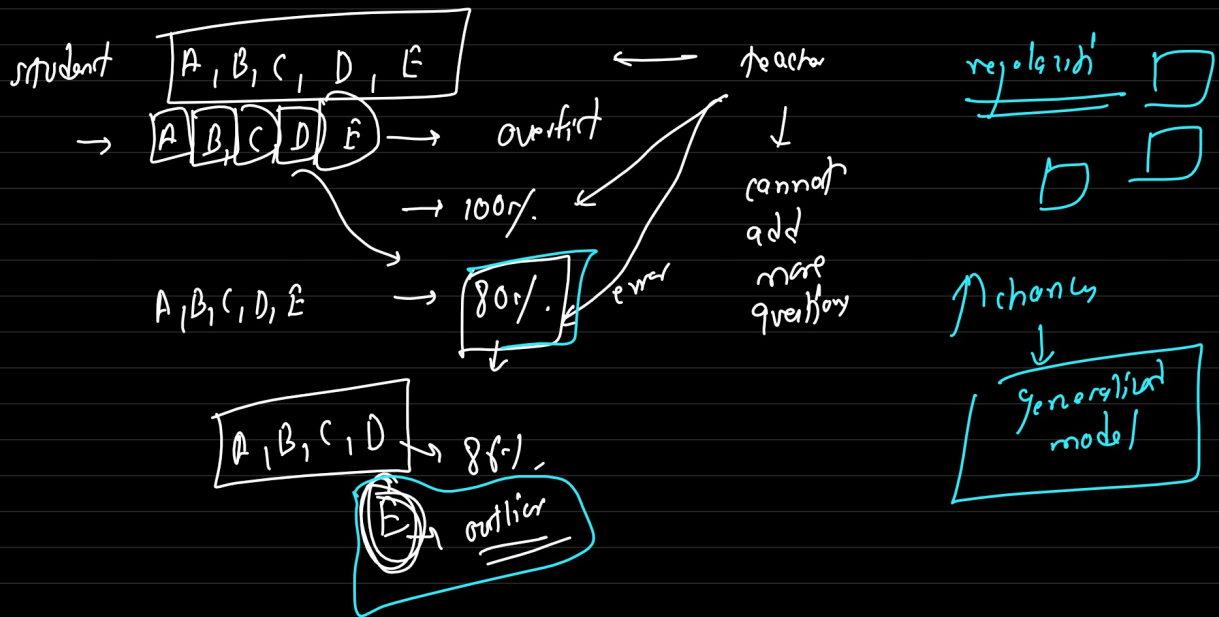
Case-2

(1) (2)
 $m_1 \rightarrow 1$ $m_1 \rightarrow 5 < 5$
 $m_2 \rightarrow 0$ $m_2 \rightarrow$
 $m_3 \rightarrow 0$ $m_3 \rightarrow$
 $c \rightarrow 0$ $c \rightarrow 0$

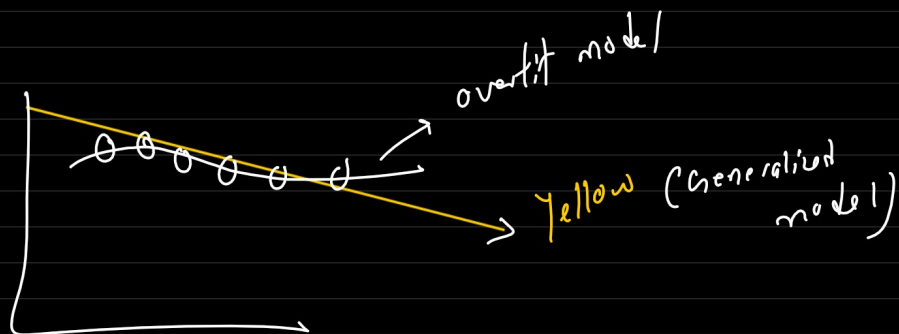
$\rightarrow$ (2)

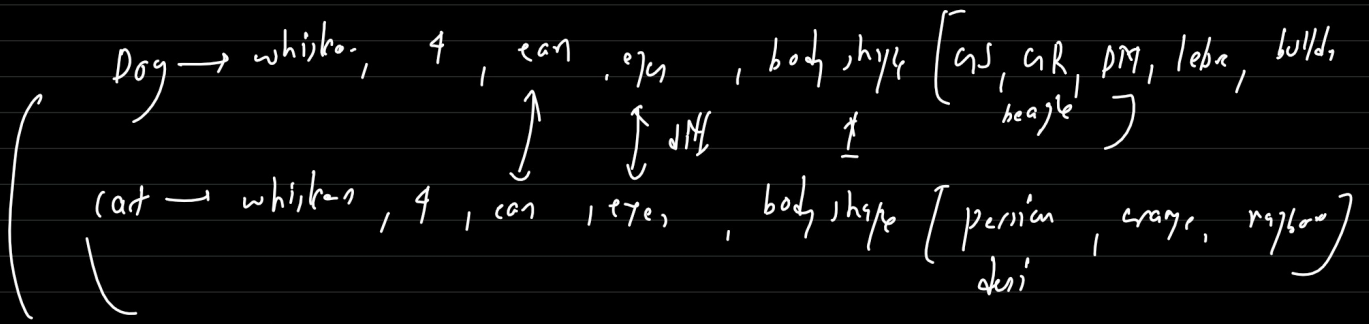


$\rightarrow$ prevent overfitting (introducing more error)
 $\rightarrow$ improve generalization



$\rightarrow$ solution
not overfit model





CV model (OL model) → too perfect

dog-CS angle → model →

error → ∞

L1 (Lasso):

$$loss_{new} = \text{Modified MSE} + \lambda \sum |\beta_i|$$

parameter →

initialisation:

$$400 + \lambda \cdot 4003$$

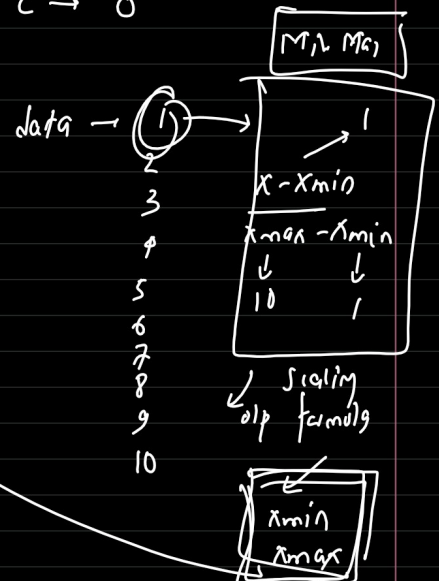
$$\beta = \begin{bmatrix} 4000 \\ 0 \\ 1 \\ 2 \end{bmatrix} \begin{matrix} m_1 \\ m_2 \\ m_3 \\ m_4/c \end{matrix}$$

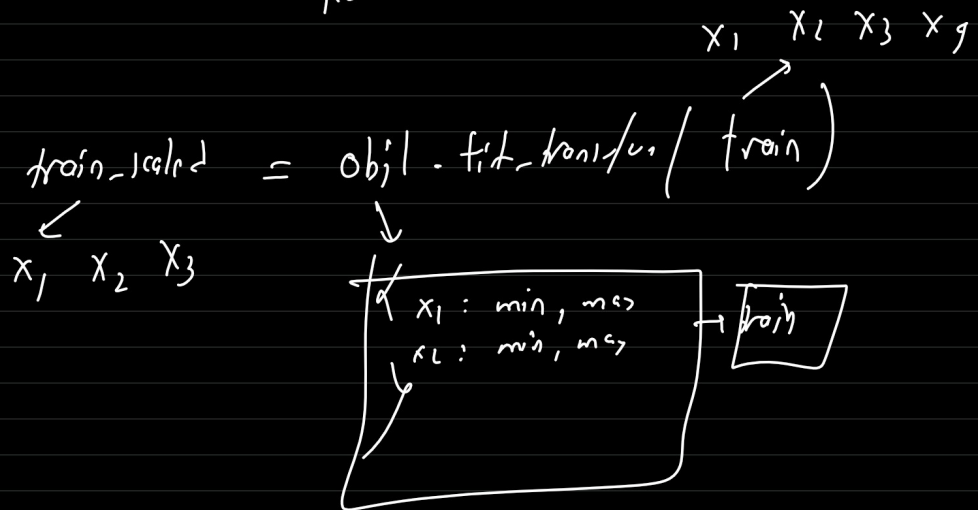
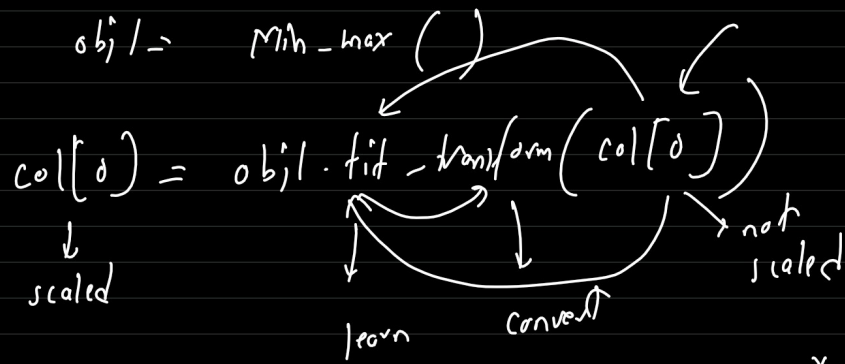
error ← $400 + \lambda \cdot 10$

$$\begin{matrix} m_1 \rightarrow 0 \\ m_2 \rightarrow 10 \\ m_3 \rightarrow 0 \\ c \rightarrow 0 \end{matrix}$$



transform





test_transform = obj1.transform(test)

~~fit~~



fit_transform()
transform()
fit()

